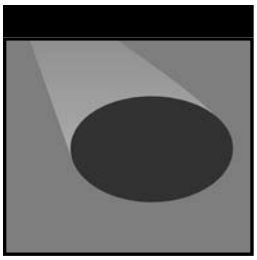


Agile Modeling:
Effective Practices for eXtreme Programming and the Unified Process

Scott Ambler

Wiley Computer Publishing

John Wiley & Sons, Inc.

Publisher: Robert Ipsen
Editor: Theresa Hudson
Development Editor: Kathryn A. Malm
Managing Editor: Angela Smith
New Media Editor: Brian Snapp
Text Design & Composition: D&G Limited, LLC

This book is printed on acid-free paper. ♾

Published by John Wiley & Sons, Inc., New York

Published simultaneously in Canada.

Library of Congress Cataloging-in-Publication Data:

ISBN: 0-471-20282-7

Printed in the United States of America.

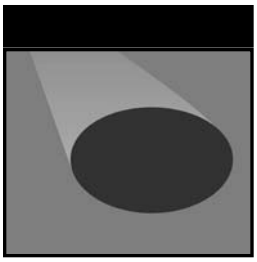
10 9 8 7 6 5 4 3 2 1

To my fellow black-belt candidates,

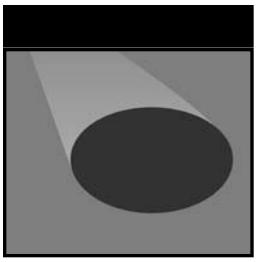
Ed Maloney, Marc Desroches, and Tyler Colisimo

We made it.

Wiley, Hoboken, ISBN: 9780471271901



Contents



Foreword

When Scott asked me to write the Foreword for this book, I was both pleased and surprised. I was pleased for several reasons: it's always nice to be thought of, it's great to be associated with a book as well-written as this one, and agile methods (specifically eXtreme Programming) are my own focus these days. I was surprised because I was one of the first and most vocal in "suggesting" that Scott *not* address his attention to this topic. "I will explain. No, it is too much. I will sum up."

Software development, to me, is best done with as little specialization as possible. I teach and believe that the best results are obtained with a team of individuals who contribute wherever and however they can, without regard to who is the architect, the modeler, the designer, the programmer, or the tester. Not that we all have to be some kind of godlike software Renaissance man or woman, but that we should all come together and contribute as much of everything as we can.

Furthermore, software development (again, to me) is best done with as little modeling as possible. The point of software development is to develop software. Too much time is taken up in getting ready, leaving too little time for the important part—actually producing solid, well-designed, high-quality software. The quality of the software isn't always correlated with the quality of the models that are drawn with the latest modeling tools before building the system.

So when Scott proposed and started his Agile Modeling forum and Web site, I was opposed to the idea. Too much concentration on modeling, I feared, would take away from the focus on bringing together the team and the skills necessary to build good software. I said so, in no uncertain terms, on Scott's forum and in private e-mails.

Well, it turns out that Scott recognized something that I did not. If people are going to come together in cooperation to build software in the agile fashion, it is not enough that they only bring love, understanding, and passion. They must have skill. The team must have skill in analysis, modeling, design, programming, testing—all of the things that are the basis of good software. And they must apply those skills consistently with the values of agile development, one of which is to focus on "individuals and interactions over processes and tools," as we wrote in the Agile Manifesto.

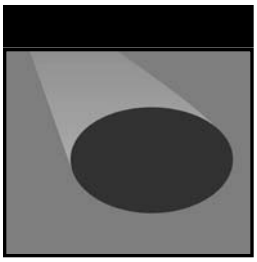
That's what Scott set out to do in his forum, with his Web site, and in this book. He shows us how to perform effective, light-weight modeling in the context of an agile project. He shows us how to do modeling in non-agile projects, with an eye to helping them become more agile. Most importantly, he does this based on a set of values, principles, and practices that can inform an individual's, and a team's, approach to modeling. He keeps his eye on the game of software development at all times, while focusing on the particular modeling skills and practices you'll need.

Scott addresses the use of simple tools, the kind of workspace required, the way the team needs to be constituted, and how it should work together. I particularly like the quote from Sergeant Rasczak of *Starship Troopers*, "I only have one rule, everyone fights and nobody quits or I'll shoot you myself." Scott relates modeling to everything from eXtreme Programming to the Unified Process, and does a good job of it.

One of one's duties in writing a foreword is to indicate who should read this book. Here's my take: read this book if you are a software developer who needs modeling skills as part of your development—that is, if you are a software developer. Read this book if you are a modeler who needs your work to be applicable to software development in these days of rapid change—that is, if you are a modeler. And read this book if you are a software development manager who needs to figure out what agile development means to your projects—that is, if you are a software development manager. If you're engaged in any way in software development today, this book can help you out.

Scott's book, *Agile Modeling*, describes the skills necessary to do effective modeling as part of software projects that need to move quickly to deliver quality software to their stakeholders. Well done, Scott!

Ron Jeffries



Preface

If you're reading this Preface, then you are very likely trying to determine whether or not you should buy this book. I like your attitude! To help you make this decision I'll quickly answer a few very important questions that you most likely have.

What Is Agile Modeling?

Agile Modeling (AM) is a practices-based process that describes how to be an effective modeler. Current modeling approaches can often prove dysfunctional. In the one extreme, modeling is non-existent, often resulting in significant rework when the software proves to be poorly thought through. The other extreme is when excessive models and documents are produced, which slows your development efforts down to a snail's pace. AM helps you find the modeling sweet spot, where you have modeled enough to explore and document your system effectively, but not so much that it becomes a burden that slows the project down.

The techniques of AM can and should be applied by project teams that wish to take an agile approach to software development, in particular those that are following an agile process such as eXtreme Programming (XP), DSDM, SCRUM, or FDD. AM can also be used to improve and often simplify your modeling efforts on projects where you don't take a purely agile approach.

What Does This Book Cover?

The book starts by exploring the values, principles, and practices of AM and describing techniques to improve your productivity as a modeler. You are likely to discover that you are already following several of these practices, although may have been afraid to admit it to others, and are likely to discover new ways to model effectively. The book also rethinks several important issues that pertain to software development, such as how to write documentation, how to organize modeling sessions and modeling teams, and where UML fits in. As the title of the book suggests, it explores in detail how to model effectively on an XP project. Contrary to what you may have heard, modeling is an important part of XP. This book shows how to simplify your modeling efforts on projects taking a Rational Unified Process (RUP) or Enterprise Unified Process (EUP) approach.

What Doesn't This Book Cover?

This book does not tell you how to create models. For example, it does not describe procedures for writing user stories, use cases, or business rules. This book is not meant to be an introduction to the UML, data modeling, or usage-centered design. This book looks at the bigger picture, at the process of modeling, not the minute details. This is very similar to XP that describes the process for developing software but not how to actually program.

Furthermore, this book is different than any other that you've read about software modeling before. Previous modeling methodology books described several modeling artifacts such as use cases, sequence diagrams, and class diagrams—and described a methodology for using these artifacts to model your software. AM takes a different approach; it describes techniques for modeling but doesn't insist on the types of models that you create. Instead it suggests that you learn how to apply a wide variety of modeling artifacts and that you strive to add more to your intellectual toolbox over time. Where other modeling methods disappear as the underlying technology changes, I believe that you will discover that the principles and practices of AM will stand the test of time because they are truly fundamental.

Who Am I?

Even though I live just north of Toronto, I'm a Senior Consultant and President of Denver-based Ronin International, Inc. I've been developing software since the mid-1980s and building object-oriented software since the early 1990s. I actively work with clients to create mission-critical software and in my spare time write about my experiences in books, magazines, and online white papers. For several years now I have focused on software process issues, including prescriptive processes such as the Unified Process (UP) as well as agile processes such as eXtreme Programming (XP), helping organizations to become more effective in their approach to software development. I

also like to take an active role on software projects, getting my hands dirty whenever I can in the role of senior developer or team lead.

Who Are You?

You are very likely a developer or modeler that wants to improve their effectiveness as a software professional. You're curious about how to model on an XP project or how to simplify your modeling efforts on a RUP project. You might even be a project manager or process expert who's trying to figure out what this "agile development stuff" is all about.

Who Helped Me?

I would like to thank the following people for their insights that have gone into this book:

Glen B. Alleman
James Ames
Dave Astels
Bruce Bacheller
Kent Beck
John Bennett
Larry Bernstein
Howard Bolling
Terry Bollinger
Grady Booch
Larry Brunelle
Scott Clemmons
Alistair Cockburn
Anthony DaSilva
Rachel Davies
Craig Dewalt
Bryan Dollery
Sara Edwards
Dale Emery
Michael C. Feathers
Rick Fisher
Peter Foreman
Martin Fowler
Martin Gainty
Adam Geras
John Goodsen
Leonard Greski
Lionell Griffith
David Hecksel
Mats Helander
Jim Highsmith
Luke Hohmann
Gerry Hummell
Ron Jeffries
Peter Lappo
Mark Levison
Dave Lubinsky
Robert C. Martin
Lynn H. Maxson
Bill Meakin
Drew Mills
John Nalbone
Miroslav Novak
Larry O'Brien

Tom Pardee
Erich Pawlik
Neil Pitman
Charlie Poole
Mary Poppendieck
Gareth Reeves
David M. Rubin
Marcelo Lopez Ruiz
Jeff Ruley
Alan Shalloway
Doug Smith
Mike Smith
Roger Smith
Jim Standley
Dr. Gernot Starke
Dan Sterling
Brian Tarbox
Dave Thomas
Neil Thorne
John Welch
Geri Winters
Klaus Wuestefeld
Ed Yourdon

Introduction to Agile Modeling

This part sets the foundation for the book through a detailed discussion of the values, principles, and practices of Agile Modeling (AM). This section includes the following chapters:

- **Chapter 1: Introduction.** This chapter outlines the challenges faced by software developers today and how Agile Software Development and Agile Modeling address those challenges.
- **Chapter 2: Agile Modeling Values.** This chapter discusses the five *values* of AM.
- **Chapter 3: Core Principles.** This chapter describes in detail the core *principles* of AM, those of primary importance to agile modelers.
- **Chapter 4: Supplementary Principles.** In this chapter we discuss the supplementary principles of AM that support and enhance AM's core principles.
- **Chapter 5: Core Practices.** This chapter presents the critical *practices* of AM, all of which you must adopt to claim that you are truly doing Agile Modeling. These practices describe how to take an incremental and iterative approach to modeling, how to support and enhance teamwork, how to keep things as simple as possible, and how to validate your efforts in an agile manner.
- **Chapter 6: Supplementary Practices.** These practices describe the motivations for creating an agile model, how to keep your documentation efforts agile, and how to improve your productivity as an agile modeler.
- **Chapter 7: Order from Chaos: How the AM Practices Fit Together.** This chapter presents an overview on how the practices fit together in a synergistic whole.

CHAPTER 1

Introduction

To change your fate, you must first change your attitude.

The current software development situation is less than ideal. Systems are regularly delivered late or over budget if they are delivered at all. Systems often don't meet the needs of our customers and we must develop them again and again and again. Our customers are angry because of these problems and are neither willing to trust us nor work with us because they've been burned too many times in the past. To make matters worse, our customers don't have a very good understanding of what we do, how we do it, or why we do it—the end result is that they put unrealistic demands on us and don't give us the support that we need to accomplish their goals.

It isn't much fun for software developers, either. We work long hours, typically 50, 60, or 70 hours a week and quickly become burned out. When projects run into trouble, often before they've even started, we point fingers at others. Convenient targets include our "pointy-haired bosses" whom we believe are barely competent enough to tie their own shoes, the "paper-pushing fools" in the department down the hall from us that demand excessive amounts of documentation, and our "stupid users" who often don't know what they want, and when they do tell us what they want, it never makes sense anyway. Naturally, we never blame ourselves; we're perfect after all. So what do we do when we realize that a project is in trouble? Sometimes we give it our all, working excessive hours in a futile attempt to meet the unrealistic demands placed on us, embarking on a death march (Yourdon 1997). Sometimes we disconnect from the project entirely. Knowing that it's doomed, we decide that we should at least learn

something useful to pad our resumes, so we download some new development tools from the Internet and start playing with them.

Why have we gotten ourselves in such a sad state of affairs? First, I believe that many people have lost sight of the fact that the primary goal of software development is to build systems, in the most effective and efficient manner possible, that meet the needs of their users. For the sake of our discussion, a system includes the software, documentation, hardware, middleware, installation procedures, and operational procedures. Similarly, organizations have lost sight of the end-to-end process of delivering software to customers and have unfortunately organized IT departments into teams with specialist roles that often don't see the overall picture. I suspect that this has happened because one, if not two, generations of IT professionals believe that they must follow a predefined set of activities in order to develop software. We can describe these activities by what is known as prescriptive processes, also referred to as heavyweight software processes. Prescriptive software processes such as the Unified Process (Kruchten 2000; Ambler 2001b), the OPEN Process (Graham, Henderson-Sellers, and Younessi 1997), and the Object-Oriented Software Process (Ambler 1998; Ambler 1999) all have their place. It is just that they may not be as appropriate as their supporters consider them to be. The problem with these approaches is that they typically focus on prescriptive procedures and the artifacts that should be created, approaches that are often implemented by organizations who consider people to be "plug and play compatible." In other words, their belief is that, with the right process in place and with the necessary number of artifacts, you can swap people in and out of a project with relative ease. My experience is that this is only true when the person you are replacing isn't very productive, something that is often the case in organizations following a heavy-weight process. The reality is that replacing a productive person is difficult regardless of the process you follow, so the "plug and play compatibility" goal is questionable at best.

The interesting thing about prescriptive processes is that they are attractive to management but not to most developers. Prescriptive processes are typically based on a command-and-control paradigm that puts management in control of things, well, at least makes them perceive that they're in control of things. It also has a tendency to make management think they can minimize the role of project stakeholders in software development, bringing them in for a few quick requirements sessions and then ignoring them until they're needed for user acceptance testing. Another problem is that when the going gets tough, developers quickly abandon the process, unfortunately throwing out the good with the bad when they do so, and then they often find themselves in an even bigger mess than before. My experience is that short cuts often lead to quagmires, not salvation, making your death march even worse.

I also believe that the way that developers learn their trade has a few unique dysfunctions. For the most part our colleges and universities are doing a reasonable job of educating developers for entry-level jobs. However, even if the schools were doing a perfect job and everyone was getting a degree or diploma, I suspect that we'd still have a problem due to the inherent nature of software developers. When software developers are young, in their teens or early twenties, they typically focus on learning and working with technology. They describe themselves as PERL programmers, Linux experts, Enterprise JavaBeans (EJB) developers, or .NET developers. To them the tech-

nology is the important thing. Because the technology is constantly changing, younger developers have a tendency to just barely learn a technology, apply it on one or two projects, and then start over again learning a new technology or the latest incarnation of what they worked with previously. The problem is that they keep learning the same different flavors of the same low-level, fundamental skills over and over again.

Luckily, many developers become aware of this after several rounds of technologies—once you've written code for transaction control in COBOL, Java, and C#, you start to realize that the fundamentals don't change. The same is true of database access in various environments, user interface design, and so on. Before long, developers begin to realize that many of the fundamentals, which they may or may not have been taught in school, remain the same regardless of the technology. This realization often comes when developers reach their late twenties or early thirties, typically the time when people start to settle down, get married, and buy a house. This is fortuitous because these new demands mean that developers can no longer afford to invest vast amounts of time learning new technologies; instead, they want to spend that time with their families. Suddenly higher-level roles such as project lead, project manager, and (non-agile) modeler become attractive to them because these roles don't require the constant and intensive effort needed to learn new technologies. So, by the time that developers begin to truly learn their craft they're in the process of transitioning out of their roles as developers. Luckily, new "young punks" come along and the cycle repeats itself. The end result is that the majority of people actively developing software are typically not the ones best qualified to do it, and they don't even know it.

Things aren't much better on the business side of things. Our customers don't understand how software is developed, which is actually quite reasonable when you stop and think about it. My experience is that few software developers could tell you how software is developed from end-to-end as well as show a reasonable understanding of the implications of various options along the way simply because software development is spectacularly difficult. Furthermore, our customers generally aren't really interested in participating in complex processes that they don't understand well, and in these situations, they'd rather leave the details to us so that they can get back to doing their jobs. They accept that their involvement is limited to being involved in a requirements workshop or two at the beginning of the project, reviewing key documents throughout the project, receiving glowing (albeit sometimes falsified) status reports throughout, getting involved with acceptance testing just before delivery, and finally receiving the system, often late and over budget. This is the way that it's always been, this is the way the IT professionals tell them it has to be, and they often don't think to question what's happening. What's strange is that they tolerate this situation. What they're asking for is software that meets their needs in an effective manner, but what the developers are giving them is a bunch of documents to review, some status reports, some tests, and then finally some software if things go well. In other words, current practice is to deliver what we want to deliver, not to deliver what customers are asking of us. As you'll see in this book, it is possible (and required) for our project stakeholders to be actively involved; we just have to ensure that the development process is acceptable to them.

Whew! I'm glad that I've gotten all that negativity out of me. Now let's take a positive approach and investigate what we can do to address these problems.

Enter Agile Software Development

To address the challenges faced by software developers, an initial group of 17 methodologists formed the Agile Software Development Alliance (www.agilealliance.org), often referred to simply as the Agile Alliance, in February 2001. An interesting thing about this group is that they all came from different backgrounds, yet were able to agree on issues that methodologists typically don't agree upon (Fowler 2001a). This group of people defined a manifesto for encouraging better ways of developing software, and then, based on that manifesto, formulated a collection of principles that defines the criteria for agile software development processes such as Agile Modeling.

The Manifesto for Agile Software Development

The manifesto (Agile Alliance 2001a) is defined by four simple value statements—the important thing to understand is that while you should value the concepts on the right side, you should value the things on the left side (presented in bold) even more. A good way to think about the manifesto is that it defines preferences, not alternatives, encouraging a focus on certain areas but not eliminating others. The Agile Alliance values are:

Individuals and interactions over processes and tools. Teams of people build software systems, and to do that they need to work together effectively with programmers, testers, project managers, modelers, and customers. Who do you think would develop a better system: five software developers with their own tools working together in a single room or five low-skilled "hamburger flippers" with a well-defined process, the most sophisticated tools available, and the best offices money could buy? If the project was reasonably complex, my money would be on the software developers, wouldn't yours? The point is that the most important factors to consider are the people and how they work together, because if you don't get that right, the best tools and processes won't be of any use. Tools and processes are important, don't get me wrong, it's just that they're not as important as working together effectively. Remember the old adage, a fool with a tool is still a fool. This can be difficult for management to accept because they often want to believe that people and time, or men and months, are interchangeable (Brooks 1995).

Working software over comprehensive documentation. When you ask a user whether they want a 50-page document describing what you intend to build or the actual software itself, what do you think they'll pick? My guess is that 99 times out of 100 they'll choose working software. If that is the case, doesn't it make more sense to work in such a manner that you produce software quickly and often, giving your users what they prefer? Furthermore, I suspect that users will understand any software that you produce much more easily than they will understand complex technical diagrams describing its internal workings or describing an abstraction of its usage, don't you? Documentation has its place; written properly, it is a valuable guide for people's understanding

of how and why a system is built and how to work with the system. However, never forget that the primary goal of software development is to create software, not documents—otherwise we would call it documentation development, wouldn't we?

Customer collaboration over contract negotiation. Only your customers can tell you what they want. Yes, they likely do not have the skills to exactly specify the system. Yes, they likely won't get it right the first time. Yes, they'll likely change their minds. Working together with your customers is hard, but that's the reality of the job. Having a contract with your customers is important, and having an understanding of everyone's rights and responsibilities may form the foundation of that contract, but a contract isn't a substitute for communication. Successful developers work closely with their customers; they invest the effort to discover what their customers need, and they educate their customers along the way.

Responding to change over following a plan. People change their priorities for a variety of reasons. As work progresses on your system, your project stakeholder's understanding of the problem domain and of what you are building changes. The business environment changes. Technology changes over time, although not always for the better. Change is a reality of software development, a reality that your software process must reflect. There is nothing wrong with having a project plan. In fact, I would be worried about any project that didn't have one. However, a project plan must be malleable, there must be room to change it as your situation changes; otherwise, your plan quickly becomes irrelevant.

The interesting thing about these value statements is they are something that almost everyone will instantly agree to, yet will rarely adhere to in practice. Senior management will always claim that its employees are the most important aspect of your organization, yet insist they follow ISO-9000 compliant processes and treat them as replaceable assets. Even worse, management often refuses to provide sufficient resources to comply with the processes that they insist project teams follow. Everyone will readily agree that the creation of software is the fundamental goal of software development, yet insist on spending months producing documentation describing what the software is and how it is going to be built, instead of simply rolling up their sleeves and building it. You get the idea—people say one thing and do another. This has to stop now. Agile modelers do what they say and say what they do.

The Principles for Agile Software Development

To help people gain a better understanding of what agile software development is all about, the members of the Agile Alliance refined the philosophies captured in their manifesto into a collection of twelve principles (Agile Alliance 2001b) that Agile software development methodologies, such as Agile Modeling (AM), should conform to. These principles are:

1. Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter time scale.
4. Business people and developers must work together daily throughout the project.
5. Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
7. Working software is the primary measure of progress.
8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
9. Continuous attention to technical excellence and good design enhances agility.
10. Simplicity—the art of maximizing the amount of work not done—is essential.
11. The best architectures, requirements, and designs emerge from self-organizing teams.
12. At regular intervals, the team reflects on how to become more effective, and then tunes and adjusts its behavior accordingly.

Stop for a moment and think about these principles. Is this the way that your software projects actually work? Is this the way that you think projects should work? Read the principles again. Are they radical and impossible goals as some people would claim, are they meaningless motherhood and apple pie statements, or are they simply common sense? My belief is that these principles form a foundation of common sense upon which you can base successful software development efforts. Furthermore, I believe that they define high-level requirements for an effective software methodology, requirements used to formulate the values (Chapter 2, "Agile Modeling Values"), principles (Chapters 3, "Core Principles," and 4, "Supplementary Principles"), and practices (Chapters 5, "Core Practices," and 6, "Supplementary Practices") of Agile Modeling.

Agile Modeling

Agile Modeling (AM) is a chaordic, practice-based methodology for effective modeling and documentation of software-based systems. The AM methodology is a collection of practices, guided by principles and values, for software professionals to apply on a day-to-day basis. AM is not a prescriptive process. In other words, it does not define detailed procedures for how to create a given type of model, instead it provides advice for how to be effective as a modeler. AM is chaordic (Hock 1999), in that it blends the "chaos" of simple modeling practices and blends it with the order inherent

in software modeling artifacts. AM is not about less modeling; in fact, many developers will find that they are doing more modeling following AM than they did in the past. AM is "touchy-feely," it's not hard and fast—think of AM as an art, not a science.

Why do we want to be effective at modeling? Because modeling is an important part of any software process. Agile software processes such as Extreme Programming (XP) (Beck 2000), SCRUM (Beedle and Schwaber 2001), and Dynamic System Development Method (DSDM) (Stapleton 1997) include modeling activities. Yes, even XP includes modeling techniques such as user stories, Class Responsibility Collaborator (CRC) models, and sketches. Contrary to what XP's detractors will tell you, XP does not abandon modeling. Instead, it minimizes modeling efforts by taking a test-first approach to design in which you develop your tests before you develop your code. This forces you to think through how you will build your software before you actually build it, exactly as traditional design modeling does. XP fulfills some of the goals of modeling, understanding what it is you're building, in different ways and therefore requires less modeling. There is absolutely nothing wrong with that. Prescriptive software processes also include modeling activities. In the case of the Unified Process (UP), three of the six core process disciplines (formerly called workflows) focus on modeling, and my own OOSP has a project stage simply called "Model."

There are two primary reasons why you model: to understand what it is you are building or to aid your communication efforts within your team or with your project stakeholders. You may choose to model the requirements of your system, perhaps with a use case diagram (common modeling artifacts are described in Appendix A of this book) or a collection of business rule definitions. Similarly, you may choose to develop models to analyze those requirements, or to formulate a high-level architecture or a detailed design for your system. In each of these cases your goal is to gain a better understanding of one or more aspects of your system. In other words, you use models to help you to explore what it is you are working on. Furthermore, you may use models to communicate within your team or with individuals or groups external to your team. A data model helps to communicate the structure of your database to people writing Java source code that interacts with that database. A user interface flow diagram communicates the overall structure of your system's user interface to the people working on individual screens, web pages, or reports. An activity diagram communicates the business processes that your system proposes to support to the project stakeholders providing funding to your project team. In short, modeling is critical to your project team's success. But how do you model in an effective and agile manner? That is the fundamental question that AM addresses.

AM has three goals:

1. To define and show how to put into practice a collection of values, principles, and practices pertaining to effective, light-weight modeling. What makes AM a catalyst for improvement aren't the modeling techniques themselves—such as use case models, class models, data models, or user interface models—but how to apply them.
2. To address the issue of how to apply modeling techniques on software projects taking an agile approach. Sometimes it is significantly more productive for a developer to draw some bubbles and lines to think through an idea, or to

compare several different approaches to solving a problem, than it is to simply start writing code. There is a danger in being too code-centric—sometimes a quick sketch can avoid significant churn when you are coding.

3. To address how you can improve your modeling activities following a "near-agile" approach to software development, and in particular, project teams that have adopted an instantiation of the Unified Process such as the Rational Unified Process (RUP) (Kruchten 2000) or the Enterprise Unified Process (EUP) (Ambler 2001b). Both of these processes are flexible enough to be tailored so that on the one extreme they are very prescriptive or on the other extreme agile enough so that AM will work with them. Although you must be following an agile software process to truly be agile modeling—more on this in a moment—you may still adopt and benefit from many of AM's practices on non-agile projects. This is similar to non-XP teams benefiting from adoption of some of its practices such as pair programming or refactoring—they aren't truly doing XP but have still improved their productivity by adopting a portion of it.

An important concept to understand about AM is that it is not a complete software process. AM's focus is on effective modeling and documentation. That's it. It doesn't include programming activities, although it will tell you to prove your models with code. It doesn't include testing activities, although it will tell you to consider testability as you model. It doesn't cover project management, system deployment, system operations, system support, or a myriad of other issues. Because AM's focus is on a portion of the overall software process, you need to use it with another, full-fledged process such as XP, DSDM, SCRUM, or UP as indicated above. Figure 1.1 depicts this concept. You start with a base process, such as XP or UP or perhaps even your own existing process, and then tailor it with AM (hopefully adopting all of AM) as well as other techniques as appropriate to form your own process that reflects your unique needs. Alternatively, you may decide to pick the best features from a collection of existing software processes to form your own process. To simplify the discussion throughout this book, I will assume that you have taken the first approach and started with a base process.

Although AM is independent of other processes, such as XP and the UP, it is used to enhance those processes. In Part Three of this book I will show how to tailor AM into XP, and in Part Four I will discuss how to tailor it into the UP.

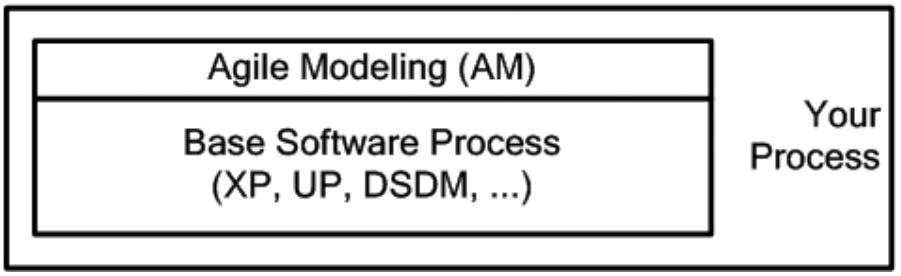


Figure 1.1 AM enhances other software processes.

Who Are Agile Modelers?

An agile modeler is anyone who follows the AM methodology, applying AM's practices in accordance with its principles and values. An agile developer is someone who follows an agile approach to software development. An agile modeler is an agile developer. Not all agile developers are agile modelers.

Now I'm going to confuse you a bit, something for which I am sorry. Throughout this book I will use the term "agile modeler" whenever I want to indicate an activity pertinent to someone taking an agile modeling approach. I'll use the more generic term "agile developer" to discuss an activity that is not only pertinent to AM but to agile software development in general. In other words, when I say that an agile developer does something, you should assume that it is something that agile modelers do, too.

A Brief Overview of Agile Modeling

In Chapter 2 you will see that AM adopts the values XP (Beck 2000), which are *communication*, *simplicity*, *feedback*, and *courage* and adds a fifth value, *humility*. It is critical to have effective communication within your development team as well as with and between all project stakeholders. You should strive to develop the simplest solution possible that meets all of your needs and to obtain feedback regarding your efforts often and early. Furthermore, you should have the courage to make and stick to your decisions, and to have the humility to admit that you may not know everything, that others have value to add to your project efforts.

AM is based on a collection of principles (Chapters 3 and 4), derived from the principles of the Agile Alliance, such as the importance of *assuming simplicity* when you are modeling and *embracing change* as you are working, because requirements do in fact change over time. You should recognize that *incremental change* of your system over time enables agility and that you should strive to obtain *rapid feedback* on your work to ensure that it accurately reflects the needs of your project stakeholders. Agile modelers realize that *software is your primary goal*, although they balance this with the recognition that *enabling the next effort is your secondary goal*. You should *model with a purpose*. If you don't know why you're working on something, then you shouldn't be doing so. You also need to recognize that you need *multiple models* in your development toolkit to be effective. A critical concept is that models are not necessarily documents, a realization that enables you to *travel light* by discarding most of your models once they have fulfilled their purpose. Agile modelers believe that *content is more important than representation*, that there are many ways you can model the same concept yet still get it right. To be an effective modeler you need to *know your models*. To be an effective teammate you should realize that *everyone can learn from everyone else*, that you should *work with people's instincts*, and that *open and honest communication* is often the best policy to follow to ensure effective teamwork. Finally, a focus on *quality work* is important because nobody likes to produce sloppy work, and *local adaptation* of AM to meet the exact needs of your environment is important.

To model in an agile manner you will apply AM's practices (Chapters 5 and 6) as appropriate. Fundamental practices include *creating several models in parallel*, *applying the right artifact(s)* for the situation, and *iterating to another artifact* to continue moving forward

at a steady pace. *Modeling in small increments* and not attempting to create a magical "all encompassing model" is also fundamental to your success as an agile modeler. Because models are only abstract representations of software, abstractions may not be accurate. You should strive to *prove it with code* to show that your ideas actually work in practice and not just in theory. *Active stakeholder participation* is critical to the success of your modeling efforts because your project stakeholders know what they want and can provide you with the feedback that you require. There are two fundamental reasons why you create models, either you *model to understand* an issue (such as how to design part of your system) or you *model to communicate* what your team is doing (or has done). The principle of *assume simplicity* is supported by the practices of *creating simple content* by focusing on only the aspects that you need to model and not attempting to create a highly detailed modeling, *depicting models simply* via use of simple notations, and *using the simplest tools* to create your models. You travel light by *discarding temporary models* and *updating models only when it hurts*. You enable communication by turning models into information radiators (Cockburn 2002), by *displaying models publicly*, either on a wall or on an internal web site, through *collective ownership* of your project artifacts, through *applying modeling standards*, and by *modeling with others*. You greatly enhance your development efforts when you *consider testability*, *apply patterns gently*, and *reuse existing artifacts*. Because you often need to integrate with other systems, including legacy databases and web-based services, you will find that you need to *formalize contract models* with the owners of those systems.

At its core, AM is simply a collection of techniques that reflect the principles and values shared by many experienced software developers. If there is such a thing as agile modeling, then are there also agile models? Yes.

What Are Agile Models?

To understand AM you need to understand the difference between a model and an agile model. A model is an abstraction that describes one or more aspects of a problem or a potential solution addressing a problem. Traditionally, models are thought of as zero or more diagrams plus any corresponding documentation. However, non-visual artifacts such as collections of CRC cards, a textual description of one or more business rules, or the structured English description of a business process are also considered models. An agile model is a model that is just barely good enough. But how do you know when a model is good enough? Agile models are good enough when they exhibit the following traits:

Agile models fulfill their purpose. Sometimes you model to communicate; perhaps you need to communicate the scope of your effort to senior management; and sometimes you model to understand; perhaps you need to determine a design strategy to implement a collection of Java classes. For an agile model to suffice, it clearly must fulfill the purpose for which it is created.

Agile models are understandable. Agile models are understandable by their intended audience. A requirements model will be written in the language of the business that your users comprehend, whereas a technical architecture model will likely use technical terms that developers are familiar with. The modeling notation that you use affects understandability—UML use case diagrams are of

no value to your users if they don't understand what the notation represents. In this case you would either use another approach or educate them in the modeling technique. Style issues, such as avoiding crossing lines, will also affect understandability—messy diagrams are harder to read than clean ones. The level of detail in your models, see below, can also affect understandability, because a highly detailed model is harder to comprehend than a less detailed one. Simplicity (see later in this chapter) is similarly a factor that affects understandability.

Agile models are sufficiently accurate. Models often do not need to be 100 percent accurate; they just need to be accurate enough. For example: If a street map is missing a street, or it shows that a street is open but you discover it's closed for repairs, do you throw away your map and start driving mayhem through the city? Likely not. You might decide to update your map. You could pull out a pen and do it yourself or go to the local store and purchase the latest version (which still might be out of date). Or you could simply accept that the map isn't perfect but still use it because it is good enough for your purposes—you can use it to get around because it does accurately model most of the other streets in your town. The reason you don't discard your street map the minute you find an inaccuracy is that you don't expect the map to be perfect, nor do you need it to be. Similarly, when you find a problem in your requirements model, or in your data model, you can choose to either update the model at that point or accept it as it is—good enough but not perfect. Some project teams can tolerate inaccuracies, whereas others can't. The nature of the project, the nature of the individual team members, and the nature of the organization will decide this. Sufficient accuracy depends on both the audience of the model and the issues that it's trying to address.

I was working on a project using Enterprise JavaBeans (EJB) (Roman et. al. 2002) technology and needed to explain how EJB's invocation of entity beans worked. In the process of doing this I drew a sketch explaining how EJB's concepts of home and remote interfaces worked and a couple of sketches walking people through the lifecycle of an entity. As I did this I forgot the exact details of bean activation and passivation, I even mislabeled the name of one of the operations, but I still got the general idea across to the audience. The audience was composed of people, not computers; therefore, they didn't require a perfect specification, they just needed a description that was good enough to provide a basic explanation from which they could fill in the blanks and correct any mistakes.

Agile models are sufficiently consistent. An agile model does not need to be perfectly consistent with itself or with other artifacts to be useful. If a use case clearly invokes another in one of its steps, then the corresponding use case diagram should indicate that with an association between the two use cases that is tagged with the UML stereotype of <<include>>. However, you look at the diagram and it doesn't! Oh no, the use case and the diagram are inconsistent! Danger, Will Robinson, Danger! Red alert! Run for your lives! Wait a minute; your use case model is clearly inconsistent, yet the world hasn't come to an end.

Yes, in an ideal world all of your artifacts would be perfectly consistent, but no, it often doesn't work out that way. When you're building a simple business application, you can tolerate some inconsistencies. For example: On a use case model you could have an actor called "Customer," yet in your class model a class called "Client." Is it customer, client, or both? If it's important, you can look into it and address the problem appropriately, but if isn't important, you can simply move on and live with the inconsistency. Granted, sometimes you can't tolerate inconsistencies—witness NASA's recent learning experience regarding the metric and imperial measuring systems when they accidentally slammed a space probe into Mars in 1999. The point is that an agile model is consistent enough and no more; you very often do not need a perfect model for it to be useful.

Regarding accuracy and consistency, clearly there is an entropy issue to consider here as well. If you have an artifact that you wish to maintain, what I call a "keeper," then you will need to invest the resources to update it as time goes on. Otherwise it will quickly become out of date and effectively useless to you. For example: I can tolerate a map that is missing one or two streets, but I can't tolerate one that is missing three quarters of the streets in my town. A data model that is missing a few recently added columns still provides very good insight into your database schema, and a deployment diagram that still indicates you're using an old version of an EJB application server is likely good enough for now. There is a fine line between investing too much and not enough effort to keep your artifacts sufficiently accurate to be effective.

Agile models are sufficiently detailed. A road map doesn't indicate each individual house on each street. That would be too much detail and thus would make the map difficult to work with. However, when a street is being built, I would imagine the builder has a detailed map of the street that shows each building, the sewers, electrical boxes, and so on in enough detail to make the map useful to him. This map doesn't depict the individual patio stones that make up the walkway to each building; once again, that would be too much detail. Sufficient detail depends on the audience and the purpose for which they are using a model—drivers need maps that show streets, builders need maps that show civil engineering details.

Consider an architecture model. Depending on the nature of your environment, a couple of diagrams drawn on a whiteboard and updated as the project goes along may be sufficient. Or perhaps several diagrams drawn using a CASE tool is what you need. Or perhaps the same diagrams supported with detailed documentation are what is required. Different projects have different needs. In each of these three examples you are in fact developing and maintaining a sufficiently detailed architecture model. It's just that "sufficiently detailed" depends on the situation.

Agile models provide positive value. A fundamental aspect of any project artifact is that it should add positive value. Does the benefit that an architecture model brings to your project outweigh the costs of developing and (optionally) maintaining it? An architecture model helps to solidify the vision to which your

project team is working, which clearly has value. But, if the costs of that model outweigh the benefits, then it no longer provides positive value. Perhaps it was unwise to invest $100,000 developing a detailed and heavily documented architecture model when a $5,000 investment resulting in whiteboard diagrams that you took digital snapshots of would have done the job.

Agile models are as simple as possible. You should strive to keep your models as simple as possible while still getting the job done. Simplicity is clearly affected by the level of detail in your models, but it also can be affected by the extent of the notation that you apply. For example: Unified Modeling Language (UML) class diagrams can include a myriad of symbols, including Object Constraint Language (OCL), yet most diagrams can get by with just a portion of the notation. You often don't need to apply all the symbols available to you, so limit yourself to a subset of the notation that still allows you to get the job done.

Therefore, the definition for an agile model is that it is a model that fulfills its purpose and no more; is understandable to its intended audience; is simple, sufficiently accurate, consistent, and detailed; and investment in its creation and maintenance provides positive value to your project.

A common philosophical question is whether source code is a model, and more important, whether it is an agile model. If you were to ask me outside the scope of this book, my answer would be yes, source code is a model, albeit a highly detailed one, because it clearly is an abstraction of your software. I would also claim that well written code is an agile model. Nevertheless, in this book I will distinguish between source code and agile models for the simple reason that I need to treat the two differently—agile models help to get you to source code.

What Is(n't) Agile Modeling?

I am a firm believer that when you are describing the scope of something, be it a system or in the case of AM a methodology, you should describe both what it is and what it isn't. The following points describe the scope of AM:

AM is an attitude, not a prescriptive process. AM comprises a collection of values that agile modelers adhere to, principles that agile modelers believe in, and practices that agile modelers apply. AM describes a style of modeling, when used properly in agile environments, that results in better quality software and faster development while avoiding over-simplification and unrealistic expectations. AM is not a cookbook approach to development—if you're looking for detailed instructions for creating UML sequence diagrams or drawing user interface flow diagrams, then you need to pick up one of the many books listed in the references section at the back of the book.

AM is a supplement to existing methods; it is not a complete methodology. The primary focus of AM is on modeling, and its secondary focus is on documentation. That's it. AM techniques should be used to enhance modeling efforts of project teams following agile methodologies such as XP, SCRUM, and Crystal Clear (Cockburn 2001b). AM can also be used with prescriptive

processes such as the Unified Process, although its success will be determined by the agility of the process.

AM is complementary to other modeling processes. Modeling methodologies such as ICONIX (Rosenberg and Scott 1999) and Catalysis (D'Souza and Wills 1999) focus on the details of how to create certain types of artifacts, explaining in detail how to go about applying artifacts such as use cases, robustness diagrams, or component models. What they don't focus on are the high-level practices for effective modeling that AM does. My experience is that AM and other modeling methodologies work very well together, and I have no doubt that we will one day see books such as "Agile Modeling with ICONIX" and "Agile Modeling with Catalysis" on the market.

AM is a way to work together effectively to meet the needs of project stakeholders. Agile developers work as a team with their project stakeholders, who in turn take a direct and active role in the development of the system. To paraphrase a well-known saying about teamwork, there is no "I" in "agile."

AM is effective and is about being effective. As you read more about AM, one of the things that should become poignant to you is AM's ruthless focus on being effective. AM tells you to maximize the investment of your project stakeholders, to create a model or document when you have a clear purpose and understand the needs of its audience, to apply the right artifacts to address the situation at hand, and to create simple models whenever you can. Do no more than the absolute minimum to suffice.

AM is something that works in practice; it isn't an academic theory. The goal of AM is to describe techniques for modeling systems in an effective manner, one that is both efficient and sufficient for the task. My co-workers and I at Ronin International, Inc. (www.ronin-intl.com) have applied many of AM's techniques for several years, techniques that we have honed at a wide range of clients in various industries. Furthermore, since February 2001 several hundred modeling practitioners on the Agile Modeling mailing list (www.agilemodeling.com/feedback.htm) have examined and discussed these techniques and found them to be effective.

AM is not a silver bullet. Agile modeling is an effective technique for improving the software development efforts of many professionals. That's it, nothing more. It isn't magic snake oil that will solve all of your development problems. If you work hard, if you stay focused, if you take AM's values, principles, and practices to heart, then you will likely improve your effectiveness as a developer.

AM is for the average developer, but is not a replacement for competent people. AM's values, principles, and practices are straightforward, many of which you have likely been following or wish you had been following for years. You don't have to walk on water to be able to apply AM's techniques, but you do need to have basic software development skills. The hardest thing about AM is that it prods you to learn a wide range of modeling techniques, a long and continuing activity. Learning to model can seem difficult at first, and it is, but you can do it if you choose to learn a technique at a time.

AM is not an attack on documentation. Agile modelers create documentation that maximizes their investment in its creation and maintenance. Agile documentation is as simple as possible, as minimal as possible, has a distinct purpose that is directly related to the system being developed, and has a defined audience whose needs are understood. Agile documentation is described in detail in Chapter 14, "Agile Documentation."

AM is not an attack on CASE tools. Agile modelers use tools that provide positive value by helping to make them more effective as developers. Furthermore, they always strive to use the simplest tool that gets the job done.

The SWA Online Case Study

Throughout this book I will present models for a common case study called SWA Online. This section provides a brief management vision for the system, the type of high-level vision statement that you might receive at the start of a project.

SWA Enterprises is a distributor of high-margin goods throughout the United States, supplying specialty retail stores with unique goods that are difficult to find elsewhere. SWA Enterprises prides itself on identifying an eclectic and ever changing mix of products. Although the company has been successful to date, it wants to expand its presence to the Internet. The following is the initial vision that senior management has for the new system, called SWA Online, which it wants developed.

SWA Online will offer the entire range of physical products sold by SWA Enterprises for now, and we may want to sell virtual products such as online music and videos at some point in the future. Our target market will remain the United States for now. At one point we considered all of North America, but we consider that too aggressive for our first release—far better to focus on our existing market and get it right before venturing into new territory. Eventually, selling products internationally is our true goal.

We'll use our current distributor, Fly-By-Night Shipping, but we're concerned that they may not be able to handle our business in the future. They have proven very effective shipping to retail stores within the United States; overnight shipping is no problem as are lower cost options such as multi-day ground shipping. We're not sure how effective they are shipping internationally, and we eventually want to not rely on a single vendor for key services such as shipping.

We believe that our system, a commercial off-the-shelf (COTS) package that we purchased several years ago, which calculates taxes and handles inventory-related functionality, should be sufficient for the first release of SWA Online, although that is something that the development must confirm.

SWA Enterprises currently employs 87 people. Major focuses of the organization include sales to retail stores, buyers focused on identifying new products to carry within our catalog, and shipping and returns. We have just hired a Vice President of Online Sales, Sally Jones, who will be responsible for building the organization required to support and operate SWA Online. Sally will be actively involved with your project team and will help you to obtain access to other business staff within SWA

Enterprises as you need them. The primary responsibility of these people is naturally their full-time, day-to-day jobs, but we have instructed them to find a way to participate with your development team as much as required.

The software process that SWA Enterprises has adopted is a stripped-down version of the EUP that has several of the practices of XP tailored into it and also includes the full collection of the practices of AM.

A Brief Overview of this Book

This book is organized into five parts:

Part 1: Introduction to Agile Modeling. The foundation for the book is set in this part through a detailed discussion of the values, principles, and practices of AM.

Part 2: Agile Modeling in Practice. This part explores critical issues such as effective communication and documentation practices, using simple tools to model, and the organizational and cultural aspects that support AM. Advice for organizing modeling work areas, modeling teams, and modeling sessions is provided and an examination of the UML is presented in light of agile development.

Part 3: Agile Modeling and eXtreme Programing (XP). This part presents a detailed discussion of how to enhance XP with the principles and practices of AM. It begins by setting the record straight regarding modeling and documentation within XP. It then focuses on modeling portions of the SWA Online case study with an XP/AM approach.

Part 4: Agile Modeling and the Unified Process (UP). This part presents a detailed examination of how to simplify modeling within the UP by following the principles and practices of AM. Once again, modeling portions of the SWA Online case study are provided.

Part 5: Conclusion. This part describes important organizational and management issues that pertain to Agile Modeling.

CHAPTER 2

Agile Modeling Values

Modeling is similar to planning—most of the value is in the act of modeling, not the model itself.

When I first read *Extreme Programming Explained* (Beck 2000), one of the most poignant things about XP for me was how Kent first defined a foundation for his methodology. He did this by describing four values: communication, simplicity, feedback, and courage. This fascinated me because he had found a way to describe some of the fundamental factors that lead to success at the software development game, and he managed to do it in such a way as to personalize it for individual developers. Bravo! Therefore, when I began putting Agile Modeling (AM) together, I decided to take an approach similar to that which Kent took with XP. I would start with a set of high-level values, define a collection of concrete principles (Chapter 3, "Core Principles") based on those values, and then formulate practices (Chapter 4, "Supplementary Principles") from those values and principles that agile modelers should apply on the job.

Because I fully agree with XP's values, I have adopted all four of them—why reinvent the value wheel when you can reuse it? However, I felt that there was something missing, something that I just couldn't put my finger on. Then one day I was working at a client site and found myself in a discussion with another developer about requirements. Our users had described how they performed their jobs, and this developer disagreed with what they outlined. He insisted that they couldn't work that way even though our users had explained to him several times that yes, indeed, they could and did. He had another way to do things, which he claimed was better, and technically it probably was, but the users simply weren't interested and wanted to do things their way (users can be strange like that). Fair enough I thought, but not for this developer. He was right, the users were wrong, even though they were arguing about user requirements. Yikes. I eventually had to

insist that we go with what the users told us, and then later spent some time mentoring him in the concept that project stakeholders, *not* developers, are the source of requirements. Although this had been a growing pain for our team, it revealed to me a value that agile modelers need: humility. Therefore, the values of AM are:

- Communication
- Simplicity
- Feedback
- Courage
- Humility

Communication

What is communication? *Merriam Webster's Collegiate Dictionary 10th Edition* defines communication as "a process by which information is exchanged between individuals through a common system of symbols, signs, or behavior." Communication is a two-way street: You both provide and gain information as the result of communication. My experience is that effective communication—between everyone involved on your project, including both developers and project stakeholders—is a requisite for software development success. Problems occur when communication breaks down on a software project. For example, a developer doesn't tell her co-workers that her code doesn't work properly, resulting in extra work on the part of another developer to track down the problem. A user doesn't explain the relative importance of their requirements, and the developers focus on low-impact features while ignoring several that are critical to your organization. Your project manager downplays the importance of having new workstations for your development team and as a result doesn't get funding for needed upgrades. Developers show project stakeholders a user interface prototype that simulates the working system, but the stakeholders mistakenly believe they are seeing the real system and insist that you deliver six months earlier than initially agreed upon. Many problems experienced by software development teams can trace their causes back to miscommunication.

When you stop and think about it, one of the primary reasons you model is to help improve communication. When your users describe a complex business process, you can help to improve your understanding of the process by sketching a data flow diagram that depicts its logic. You can often learn more in five minutes drawing a diagram with your users than you can in five hours discussing it or reading about it in corporate manuals. When you try to explore the design of a portion of your system with another developer, you may choose to model portions of it together, perhaps drawing a UML class diagram to understand the structure of your classes. Together you will work through the design, talking about the implications of various approaches and negotiating how you intend to build it. Modeling helps you to communicate your ideas, to understand the ideas of others, to mutually explore those ideas to eventually reach a common understanding. In Chapter 8, "Communication," I explore the importance of communication within the software development process and how it is enhanced by AM's practices.

Simplicity

I believe you would be hard pressed to find a software engineering book that didn't mention the KISS rule (Keep It Simple Stupid!) at least once. The IT industry has preached simplicity for years, but for some reason the choir hasn't been listening. These same software engineering books then advise you to take actions that invariably complicate your software, making it harder to develop, test, and maintain. Common complications include:

Applying complex patterns too soon. If you need to implement telephone numbers for your customers, applying the Contact Point analysis pattern (Ambler 2001a) is very likely overkill. Yes, implementing this pattern is interesting, but its class hierarchy is much harder to build, test, and maintain than a simple telephone number class. You're better off waiting until you also need to implement email addresses and surface addresses to justify application of the Contact Point pattern. Don't get me wrong, I'm a firm believer in patterns, and I am often the first to admit that sometimes applying a pattern is in fact the simplest approach available to you. The point is that this isn't always the case.

Over-architecting your system to support potential future requirements. Maybe the bank information system you are currently working on may one day need to support the selling of insurance policies, so wouldn't it be better to architect a generic way to deal with financial instruments? Yes, modeling that would be very interesting, but you would be making your software more complicated than it needs to be today. Developers who are insecure about their ability to handle future change, or whose egos motivate them to produce the "ultimate" system, will often architect their software. Agile developers do not overbuild their software. They bet that they can build it as simply as they possibly can today and then add new functionality, once again as simply as possible, when they actually need it. How does this work? Martin Fowler (2001b) describes it best with his discussion of the YAGNI (*You Ain't Gonna Need It*) principle. He points out that you can't always predict what you'll need in the future and you're just as likely to be wrong about it as you are to be right. Therefore, why develop, test, and maintain that extra functionality, functionality that you definitely do not need right now and may never need? Wouldn't it be better to focus on fulfilling the current needs of your project stakeholders today, thereby keeping them happy (always a good thing), and instead have the courage to assume that you can solve tomorrow's problems tomorrow? If you focus on building the simplest thing possible today, then when you go to add new functionality tomorrow, you know that you'll be working with a simple system. Wouldn't this system be as simple to modify tomorrow when you actually need the new functionality as it is to modify today when you don't know yet if you need it?

To develop a complex infrastructure. A common mistake that project teams make is to invest the first portion of their project developing their infrastructure—typically components, frameworks, and class libraries—that they intend to use as the building blocks for their system. The idea is that their initial investment will pay off sometime down the road. However, this approach has several serious drawbacks. First, you're investing your project stakeholders' resources

without giving them something in return that they can actually use. Your stakeholders have likely asked you for specific features to help them do their jobs, and the first thing that you deliver is an error-handling subsystem. The result is that you're putting your project at risk by not delivering useful functionality quickly. Second, you're very likely ignoring the YAGNI principle and developing features in your infrastructure that you very likely won't need. A better approach is to simply develop your infrastructure as you need it throughout the project. For example: Build the error-handling subsystem over time when you discover that you actually need a subsystem to do so.

So what does all this have to do with agile modeling? The fundamental point is to keep your models as simple as possible, model today to meet today's needs, and worry about tomorrow's modeling needs tomorrow. In other words, don't over model—follow the KISS rule and not the KICK (Keep It Complex Kamikaze) rule.

TIP Don't "What if" Yourself to Death

I often see software developers divert themselves from their true task, to build software that meets the needs of their users in an effective manner, with wacky "what if scenarios." They start to over-model their software to meet all imaginable problems, problems that their project stakeholders very likely aren't concerned with or believe are so unlikely to happen that they are willing to go at risk on them. Then because they over-model, they also overbuild it. Yes, you don't want to be completely naïve in your efforts. It's reasonable to expect that some problems such as database problems or network glitches do occur, but you need to be realistic when you're modeling.

Models are critical for simplifying both software and the software process—it's much easier to explore an idea, and to improve upon it as your understanding increases, by drawing a diagram or two instead of writing tens or even hundreds of lines of code.

Feedback

The only way that you can determine whether your work is correct is to obtain feedback, and that includes feedback regarding your models. Models are abstractions. For example: A collection of use cases is an abstraction of how people will work with your system, whereas a component model is an abstraction of the internal structure of your software. How can you know if your abstractions are correct? There are many ways that you can obtain feedback regarding a model:

Develop the model as a team. Software development is a lot like swimming; it's dangerous to do it alone. When you work with other people, you quickly obtain feedback about your ideas.

Review the model with your target audience. Ideally, members of your target audience should be involved with the development of the model. Requirements

models should be developed in conjunction with your users; detailed design models should be developed with the people who will be doing the programming. If this isn't possible, then the next best alternative is to sit down with them and walk them through your models, perhaps working through usage scenarios. Informal reviews can occur on an informal basis; perhaps you hold a quick meeting with someone to get their feedback regarding your work, whereas formal reviews (Gilb and Graham 1993) take more effort to organize.

Implement the model. The surest way to obtain feedback is to implement your model in software and have your project stakeholders work with the software. The proof is in the pudding.

Acceptance testing. Fundamentally your models should reflect your project stakeholders' requirements for your system. Your stakeholders validate those requirements during acceptance testing, so by implication you are also validating your models.

It is interesting to note the timescale for each feedback technique. When you work together as a team, feedback is relatively immediate, on the scale of seconds or minutes. With informal reviews, feedback can occur within minutes or hours, although if someone is available for an informal review then, why aren't they available to work on the model to begin with? On the other hand, formal reviews may not occur for days, weeks, or even months depending on the availability of the reviewers. With implementation, feedback ideally occurs within several hours or at least days (remember, you're taking an agile approach to development). Acceptance testing typically provides feedback weeks or months later.

Why is timescale important? Because the more immediate the feedback is, the less likely your models are to deviate from what you actually need. Although all forms of feedback have their place, given the opportunity you should prefer the feedback provided through teamwork because it is immediate. Having said that, there is something to be said about proving your models with code because everything looks good on paper until you actually try it out.

Courage

Courageous people walk through the door;
they don't stand there and wonder what's on the other side.

-Sempai Rick Micucci

Agile Modeling, and agile software development in general, is new to most people as well as to the organizations in which they work. As you saw in Chapter 1, "Introduction,", agile software development principles challenge the status quo, and that's threatening for many people. It is a lot easier to sit back and accept the current situation, to not try to improve things, or to wait until someone else comes along and fixes things. This paralysis by fear is a primary contributor to the current sad state of the IT industry.

I once worked for an organization whose data administration group had a death grip on the software development group. Developers couldn't do work with corporate data without first going through the data group, they couldn't set up their own development database without going through the data group, and they certainly couldn't release software into production without the blessing of the data group. This wouldn't have been bad if the data group worked effectively, but unfortunately that wasn't the case. My team was working on an Enterprise JavaBeans (EJB) project, the first one for this company, using an Oracle database on the backend. When we heard that it would take several weeks to have someone from the data group set up our development database, we decided to do it on our own, initially spending a couple of hours doing so and then spending time over the next few days to tweak things as we needed. The reason the data group would have taken several weeks is they insisted on following their own procedures to size the database (work we had already done), ensure that our machines had sufficient disk storage (we had already maxxed out the box), and of course, fill out a myriad of forms that nobody but them wanted. They were incensed, and the manager of the group chewed out my manager, who then chewed us out. We held our ground, saying that we didn't have the time to cater to the bureaucratic whims of the data group. Nobody had successfully stood up to this group. People on my team were worried, but our priority was to build our system on time, so we fought it out. This took courage. To smooth things over, we said that we would be more than happy to work with them to administer our database and to evolve our data schema over time, which was completely true. This was where we ran into our second problem. Instead of providing us with a single database administrator (DBA), they instead wanted to put several people on our team: one to administer the database, one to work on a logical data model (which has absolutely no value when developing object-oriented software), and one to work on a physical data model (we wanted this). Furthermore, they wanted to do this in parallel with our development activities, instead of as an active part of the team. In other words, most of what they proposed was make-work. It would have been more of an effort for us to work with this separate group of people than it would have been to do the physical data modeling ourselves. We had several people more than qualified to do this, and to generate and apply the schema to the database (something several of us had done for years). Once again we fought it out with them, another courageous act considering nobody had ever done so in the past and they were already gunning for us. Everything eventually came to a head in a big meeting involving our managers and their managers. We actually had a good relationship with several of the DBAs within the data group who agreed with us privately and were hoping that we could break the deadlock within the organization. At the meeting, the head of the data group claimed that our concerns regarding our productivity slowing down were unrealistic, that his people could quickly get us moving forward in several weeks and continue helping us for the next few months. It was at that point that I said we had our database up and running and offered to show the working database to anyone in the room. We had the courage to do the right thing, to stand up to a clearly dysfunctional group, and to show everyone involved that there was a different way to do things. It was hard but in the long run, not only did our project benefit but the whole organization did, because it gave other teams the courage to stand up to the data group as well, eventually motivating them to streamline (albeit not enough in my opinion) their processes.

Agile methodologies ask you to closely work with other people, to trust them, and to trust yourself. This takes courage. Methods such as XP and AM ask you to do the simplest thing that you can, to trust that you can solve tomorrow's problems tomorrow. This takes courage. AM asks that you create documentation only when you absolutely need it, not just when it feels comfortable to do so. This takes courage. XP and AM ask that you let business people make business decisions, such as requirements' prioritization, and let technical people make technical decisions, such as how the software will fulfill individual requirements. This takes courage. AM asks that you use the simplest tools possible, such as whiteboards and paper, and that you use complex modeling tools only when they provide the best value possible. This takes courage. AM asks that you not dally making your diagrams pretty in order to put off difficult tasks such as proving your models with code. This takes courage. AM asks that you trust your co-workers, trust that programmers can make design decisions, and therefore you do not need to provide them with as much detail. This takes courage. AM asks you to choose to succeed, to end the cycle of near-disasters and outright failures within the IT industry. This takes courage.

I believe that courage is a fundamental requisite of agile software development. First, courage is important because you need to choose to take an agile approach, and then stick with that approach when the going gets tough (and it always does). There will be people in your organization with other visions—you need to adopt the XYZ tool, you need to adopt a heavy-weight process, management needs to exert greater control, you need to outsource all of IT, you need to follow the dictates of this other department—and they will fight your efforts every step of the way. That's what politics are all about. Second, during development you need courage to make important decisions, such as choosing one architectural approach over another or deciding which development language to work in. During development you also need courage to change direction when some of your decisions prove inadequate, by either discarding or refactoring your work.

Third, you need courage to recognize that you're fallible and will make mistakes.

Fourth, you need courage to trust that you can overcome tomorrow's problems tomorrow. Courage, it's not just for breakfast anymore.*

Humility

The best developers have the humility to recognize that they don't know everything. Frankly, it isn't possible. You could be the best Java coder there is and still not know every single detail about every single Java API. Furthermore, just because you're a great Java coder, it doesn't mean that you're a great user interface designer, or a great database designer, or a great musician—it just means that you're a great Java coder. Just because you're a great Java coder, it doesn't mean that you can't learn something new from other Java coders, including the junior person on your team. In fact, I often learn more from junior people than I do from senior people, because the junior will ask

*This is a modification of a North American advertising campaign from the mid-1990s—Eggs aren't just for breakfast anymore.

me why things work and will likely challenge my own beliefs with new ways of doing things.

Agile modelers understand that their fellow developers and their project stakeholders have their own areas of expertise and have value to add to a project. Some developers will be better at coding than you are, or better at testing, or better at requirements modeling, or better at architectural modeling. Your users likely understand various aspects of the business better than you do: Senior managers have a better grasp of where their industry is headed, and operations staff know what may or may not work in production. Agile modelers have the humility to admit that they need help to do their jobs successfully, to work together with others.

Humility also comes into play in the way that you interact with people. Agile modelers have the humility to respect the people that they work with, realizing that others likely have different priorities and experiences than they do and therefore will have different viewpoints. They don't denigrate their managers by calling them "pointy-haired bosses," people in other departments "paper pushers," or their users/customers "stupid" or "totally screwed up." Denigrating people isn't an act of humility; it's an act of arrogance. Arrogance leads to communication problems that in turn lead your project into trouble, because it motivates people to stop collaborating with you. Agile modelers are humble and more effective as a result.

Beyond Motherhood and Apple Pie

Agile modelers are courageous: They seek out simplicity, they seek feedback, they communicate well, and most of all, they're humble. It almost seems that if you become an agile modeler, you'll be nominated for sainthood. The reality is that agile modelers are people. They're fallible, they have other concerns in their lives beyond effective modeling practices, and they think and act of their own accord. The reason I defined these values is not to get overly sappy, I may not have achieved this goal, but instead, to build a foundation from which individuals may build an agile mindset, and teams and organizations may build a culture that supports agile and effective development efforts. These values are also used, along with the values and principles of the Agile Alliance (2001a, 2001b) presented in Chapter 1, as a base from which to define the principles of AM in Chapters 3 and 4.

CHAPTER

3

Core Principles

Principles only mean something when you stick to them when the going gets tough.

Agile Modeling's values (Chapter 2, "Agile Modeling Values")—communication, simplicity, feedback, courage, and humility—in combination with the values and principles of the Agile Alliance (2001a, 2001b), are used to define AM's principles. When applied on a software development project, these principles set the stage for a collection of modeling practices (Chapters 5, "Core Practices," and 6, "Supplementary Practices"). AM's values, while important, are somewhat abstract and too high-level to provide much guidance to your software development efforts; hence the need for AM's principles—concepts that are far more concrete. Some of the principles have been adopted from eXtreme Programming (XP) (Beck 2000), which in turn adopted them from common software engineering techniques. Reuse! For the most part, the principles are presented with a focus on their implications to modeling efforts and as a result, material adopted from XP may be presented in a new light.

This chapter overviews AM core principles; principles that you must adopt in full to be truly able to claim that you are agile modeling (more on this later). Chapter 4, "Supplementary Principles," overviews AM's supplementary principles that define important concepts that help to enhance your modeling efforts. AM's core principles are:

- Software is your primary goal
- Enabling the next effort is your secondary goal
- Travel light
- Assume simplicity

- Embrace change
- Incremental change
- Model with a purpose
- Multiple models
- Quality work
- Maximize stakeholder investment

Software Is Your Primary Goal

The primary goal of software development is to produce high-quality software that meets the needs of your project stakeholders in an effective manner. This principle is effectively a re-wording of the Agile Alliance's (2001b) principle that "working software is the primary measure of progress." Like it or not, the primary goal is not to produce extraneous documentation, extraneous management artifacts, or even to produce models. Creating extraneous documentation can be comforting because you can fool yourself into believing that you are making progress when in fact you're not. Instead, you're actually avoiding a difficult task, likely writing and testing code that may show that your chosen approach isn't working as well as you thought it would. Writing status reports, trumpeting your successes to everyone, or even worse, covering up your failures, may make you feel good, but it isn't getting you any closer to your end goal. Have the courage to focus on what is important, the creation of a system for your users. We're not documentation developers, or even model developers; we're software developers. Think about it. Any activity that does not directly contribute to the goal of producing quality should be questioned and avoided if it cannot be adequately justified.

Enabling the Next Effort Is Your Secondary Goal

Your project can still be considered a failure even when your team delivers a working system to your users—part of fulfilling the needs of your project stakeholders is to ensure that your system is robust enough so that it can be extended over time. As Alistair Cockburn (2001b) likes to say, when you are playing the software development game, your secondary goal is to set up to play the next game. Your next effort may be to develop the next major release of your system or it may simply be to operate and support the current version that you are building. To enable it, you will not only want to develop quality software but also create just enough documentation so that the people playing the next game can be effective, transfer skills from your developers to others, motivate existing staff to stay and develop the next release of your system, or simply motivate team members to stay with your organization. Factors that you need to consider include the nature of your developers, the nature of the next effort itself, and the importance of the next effort to your organization. In short, when you work on your system, you need to keep an eye on the future. This principle supports the AM value of Communication.

Travel Light

Traveling light means that you create just enough models and documentation to get by. Every artifact that you create, and then decide to keep, will need to be maintained over time. This includes models, documents, and project management artifacts such as schedules, test suites, and source code. For example: You decide to keep seven models. Whenever a change occurs—such as a new or updated requirement, your team takes a new approach, or adopts a new technology—you will need to consider the impact of that change on all seven models and then act accordingly. If you decide to keep only three models, then you clearly have less work to perform to support the same change, making you more agile because you are traveling lighter.

Similarly, the more complex/detailed your models are, the more likely it is that any given change will be harder to accomplish (the individual model is "heavier" and is therefore more of a burden to maintain). Every time you decide to keep a model, you trade off agility for the convenience of having that information available to your team in an abstract manner (hence potentially enhancing communication within your team as well as with project stakeholders). Never underestimate the seriousness of this trade-off. Jim Highsmith (2000) points out that someone trekking across the desert will benefit from a map, a hat, good boots, and a canteen of water. They likely won't make it if they burden themselves with hundreds of gallons of water, a pack full of every piece of survival gear imaginable, and a collection of books about the desert. However, it is possible to travel too light—clearly, it would be foolish to try to cross a desert without a minimum of supplies. Similarly, a development team that decides to develop and maintain a detailed requirements document, a detailed collection of analysis models, a detailed collection of architectural models, and a detailed collection of design models will quickly discover they are spending the majority of their time updating documents instead of writing source code. A good rule of thumb is to not maintain a model until it is very clear that you need it.

You need good communication among your team to be able to effectively travel lightly; if developers don't understand your requirements or your architectural approach, or at least if there is no one that they can work with to get their questions answered readily, then you are in serious trouble. Clearly, good communication is a requisite to support traveling light. To travel light requires courage, to trust that you're not going to need a certain artifact but are prepared to create it if you are proved wrong in your assumption. Traveling light enables simplicity in your approach to development because your artifact maintenance efforts during development are dramatically decreased.

Assume Simplicity

As you develop, you should assume that the simplest solution is the best solution, and as the title suggests, this principle is clearly derived from AM's value of *Simplicity* as well as the Agile Alliance's principle of simplicity (2001b). As Kent Beck (2000) points out, the vast majority of the time the simplest solution works well, and because it is simple, it is easy to implement. The advantage is that you aren't investing extra time implementing difficult solutions, approaches that take more time and effort to put in place. The advantage is that in the few times where the simplest solution proves not to

work, you have time to implement a more difficult solution because you haven't wasted resources elsewhere. Furthermore, the simplest solution is also the easiest to maintain and to enhance.

An implication of this principle is that you don't want to overbuild your software, or in the case of modeling, don't depict additional features in your models that you don't need today. Don't over-model your system today; model based on today's existing requirements and then refactor your system in the future when your requirements evolve. The implication is that you should keep your models as simple as you possibly can. This principle is the Occam's Razor of modeling—when in doubt take the simplest approach possible.

Will taking the simplest approach work every time? Likely not, but it will work the vast majority of the time. When it doesn't work you will have learned something and will very likely have failed very early in your efforts. Contrast that to taking a complicated approach—complicated approaches fail as well—where you've invested significant resources to discover that your ideas didn't work.

Embrace Change

Accept the fact that change happens. Revel in it. Change is one of the things that make software development exciting. Requirements evolve over time. Your project stakeholders' understanding of their requirements changes over time. Project stakeholders can change as your project moves forward, new people are added, and existing ones leave. Project stakeholders can change their viewpoints as well, potentially changing the goals and success criteria for your effort. Furthermore, your business and technological environments change as your project evolves; things occur that are often beyond the scope of your control. The implication is that your project's environment changes over time.

Agile modelers embrace change. They understand that change is a common occurrence on software projects. The Agile Alliance (2001b) advises that you welcome changing requirements even late in the project lifecycle. Agile modelers know that their work will be affected by changes; they actively strive to communicate with their project stakeholders, to seek their feedback, so they can identify changes and then act accordingly. They do not blame their project stakeholders for change; instead they actively work with them to understand and communicate the implications of the changes to enable their stakeholders to make effective decisions as to if, how, and when the change will be supported by their development efforts. Furthermore, agile modelers understand that their models are only models, that developers will tear them apart and reassemble them into something better; they accept that their work will be improved upon by others.

Agile modelers also recognize an inherent danger in embracing change—the tendency to get sloppy when doing up-front work such as requirements modeling. Why invest a lot of time understanding requirements if they're only going to change? Far better to simply bang some code out and wait for your project stakeholders to tell you to change it? Right? *Wrong!* You'd be much better off investing the time to understand the requirements to the best of your ability now and implement software based on those requirements. Some requirements will change, and you need to embrace this fact, but many requirements won't change (at least not soon). Note that AM's princi-

ples of *Quality Work* and *Maximize Stakeholder Investment* counteract the tendency to get sloppy.

Incremental Change

To embrace change you need to take an incremental approach to your own development efforts, to change your system a small portion at a time instead of trying to get everything accomplished in one big release. You can make a big change as a series of small, incremental changes. In fact, the Agile Alliance's (2001b) third principle states that you should deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter time scale. An important concept to understand when agile modeling is that you don't need to get everything right the first time. In fact, it is very unlikely that you could do so even if you tried. Furthermore, you do not need to capture every single detail in your models; you just need to get it good enough at the time. It is futile to try to develop an all-encompassing model at the start of your project. Instead, put a stake in the ground and develop a small, detailed model, or perhaps a high-level model, and evolve it over time (or simply discard it when you no longer need it) in an incremental manner. It takes humility to accept that you can't get it right the first time, or even the nth time, and courage to admit it. Kent Beck (2000) said it well, "Make it run, make it right, then make it fast."

Model with a Purpose

If you cannot identify why and for whom you are creating a model, then why are you bothering to work on it at all? Many developers worry about whether their artifacts—such as models, source code, or documents—are detailed enough or if they are too detailed, or similarly if they are sufficiently accurate. What they're not doing is stepping back and asking why they're creating the artifact in the first place and whom are they creating it for. This requires humility; you aren't modeling solely for the personal satisfaction of modeling; instead you are modeling to fulfill the needs of your project stakeholders.

What are valid reasons to create a model? Often you need to understand an aspect of your software better, perhaps you need to communicate your approach to senior management to justify your project, or perhaps you need to create documentation that describes your system to the people who will be operating, maintaining or evolving it over time. You model to understand, or you model to communicate.

The following aren't valid reasons:

- Your prescriptive process tells you to, so you dutifully do so without considering whether it makes sense to.
- Someone else has requested the model but is unable to explain why they need it, other than because they told you to create it.
- Instead of having a face-to-face conversation with someone, and have the option to do so, you instead want to create a model to give to them.

Your first step when modeling is to identify a valid purpose for creating a model and the audience for that model. Once you identify the purpose and audience, develop the model to the point where it is both sufficiently accurate and sufficiently detailed. Once a model has fulfilled its goals you should stop working on it—you are finished! Move on to something else, such as writing some code to show that your model works. This has the advantage that your models will remain simple because you won't clutter them with needless detail. This principle also applies to a change to an existing model: If you make a change such as applying a known pattern, then you need to have a valid reason to make that change, such as to support a new requirement or to refactor your work to something cleaner. An important implication of this principle is that you need to know your audience, even when that audience is yourself. For example: If you create a model for maintenance developers, what do they really need? Do they need a 500-page comprehensive document or would a 10-page overview of how everything works be sufficient? Don't know? Go talk with them and find out.

Another way to look at it is this: The point at which a model just barely fulfills its purpose is also the point of diminishing returns for that model. When you first work on a model you most likely have a sense of accomplishment because you're thinking something through, gaining a better understanding of what you need to do, or gaining improved insight into how you should build it. As you continue to work, you get closer and closer to your goal for developing that model, whatever that goal is, until you finally get to the point where you've reached your target. It's at this point that further work on the model is providing less and less benefit. Yes, you may be filling in some details. Yes, you may be improving its consistency and accuracy, but you could have moved on; you could have found something else to work on, ideally, source code that provides greater benefit to your project.

Multiple Models

You have a wide range of modeling artifacts, many of which are summarized in Appendix A, "Modeling Techniques," available to you. These artifacts include the diagrams of the Unified Modeling Language (UML) (Object Management Group 2001), structured development artifacts such as data models, and low-tech artifacts such as essential user interface models. Each artifact has its strengths and weaknesses; each one is appropriate for some situations and not others. Because modern software is complex, no single artifact, even in the case of the UML family of artifacts, is applicable for all situations. The implication is that to be effective, you need to use multiple models to describe software systems, because each model describes a single aspect of your software. For example: Figure 3.1 shows the logic for placing an order online, whereas the user interface (UI) flow diagram of Figure 3.2 describes how users navigate around the UI of SWA Online. It is interesting to note that the sequence diagram is an artifact prescribed by the UML, whereas the UI flow diagram is not (yet), and that both diagrams depict different but important aspects of the SWA Online system.

By using each model for what it is good for, and not trying to use them when it isn't appropriate, you can describe the complexities of what you are building using several

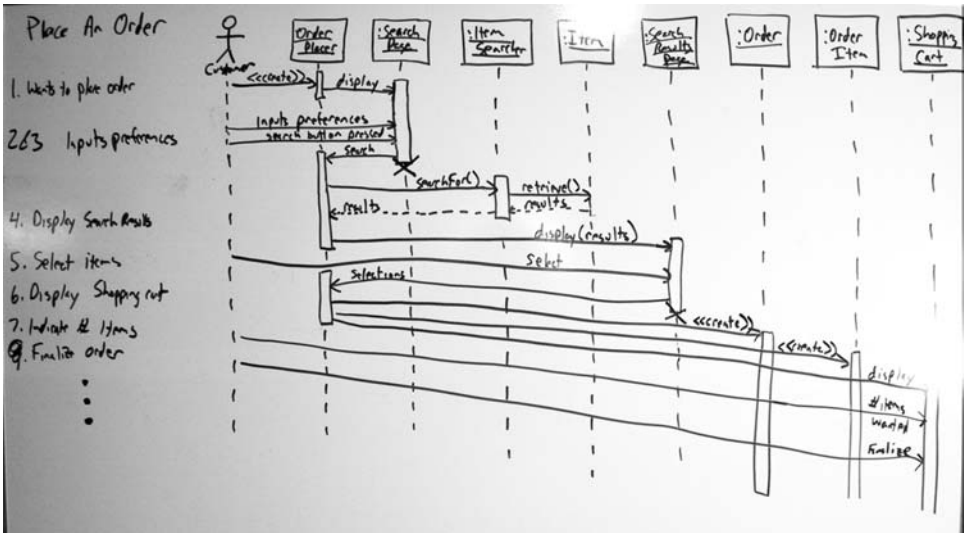


Figure 3.1 A UML sequence diagram for placing an order.

simple models instead of one or two very complex ones. This is easier for you as a developer, when you're working on the database for your system use data models, when you're working on the UI use UI-oriented models such as user interface flow diagrams. It's also easier for anyone that you need to communicate with because they can focus on one model at a time instead of trying to understand everything at once. This principle clearly supports AM values of *Simplicity* and *Communication*.

Note that although you have a wide range of models available to you, you don't need to develop all of them for any given system. Depending on the exact nature of the software you are developing, and the software process that you are applying AM with, you will require at least a subset of the models. For example: An XP project team will apply user stories as its major requirements modeling artifact, whereas a Unified Process project team will likely apply a combination of use cases, business rules, constraints, and technical requirements. An EJB application will need object-oriented design artifacts such as those described by the UML, whereas a data warehouse project will need data models. Different types of projects require different subsets of artifacts. To be effective as an agile modeler you will need to learn a wide range of models to be able to apply the right type of model based on your situation. To continue to be effective you need the *Humility* (another AM value) to admit that you can always learn new techniques, often from junior developers fresh out of school or from project stakeholders who understand the business.

You Need an Intellectual Toolbox of Techniques

An analogy that I use throughout this book is that developers should have an intellectual toolbox (McConnell 1993) of techniques that they can apply when needed, just as

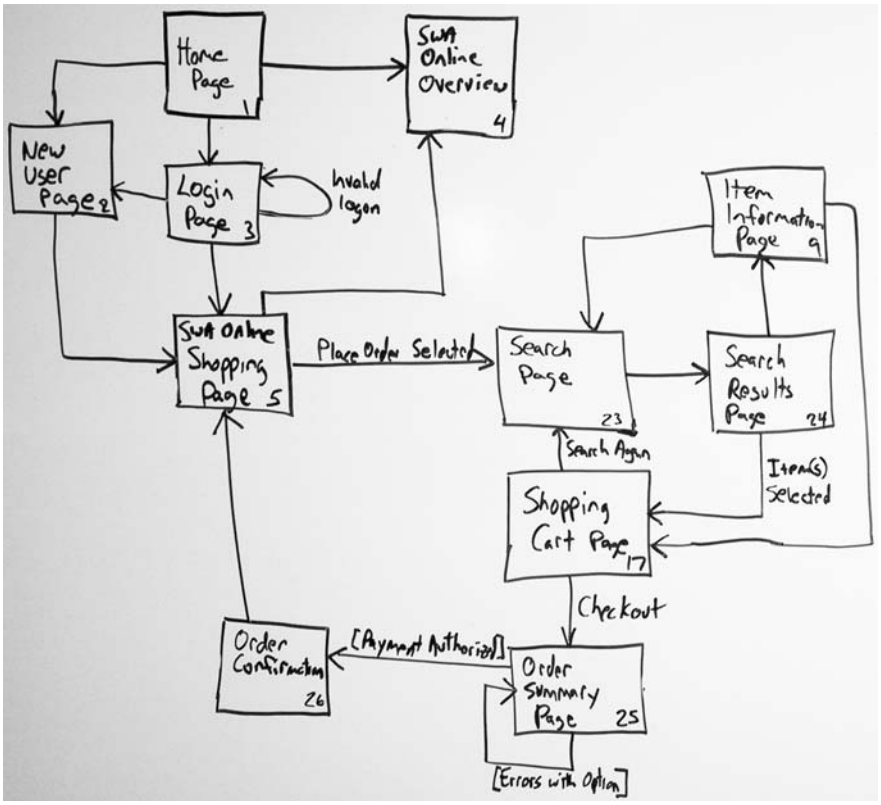


Figure 3.2 A user interface flow diagram for SWA Online.

a carpenter has a toolbox of tools. The more tools you have, and know how to apply, the more effective you will be as a developer, because you are more likely to have the right tool for the job when the need arises. Just as every fix-it job at home doesn't require you to use every tool available to you in your toolbox, every development task won't require you to apply each technique that you know. The variety of fix-it jobs you perform at home will require you to use each of your tools at some point, and similarly, the development projects you are involved with will, over time, require you to apply all of the various modeling techniques that you know. Finally, just as you use some tools more than others, you will apply some types of models more than others.

Quality Work

Nobody likes sloppy work. The people doing the work don't like it because it's something they can't be proud of. The people coming along later to refactor the work, perhaps yourself a few weeks later when your requirements change or perhaps a maintenance developer who has been assigned to evolve your system, don't like it because sloppy work is harder to understand and therefore harder to update. In other

words, quality work improves communication on your project. Your end users won't like your sloppy work because it's typically fragile and/or doesn't meet their expectations. Senior management won't like your sloppy work because they will feel they aren't getting good value for their investment in your efforts.

Agile developers understand that they should invest the effort to make permanent artifacts, such as source code, user documentation, and technical system documentation (documentation is described in detail in Chapter 14, "Agile Documentation") of sufficient quality. It takes guts to stand up and say that you need the time to do a good job. Similarly, agile developers don't invest much effort in artifacts that they intend to discard, particularly sketches or low-fidelity artifacts such as essential user interface prototypes. In other words, they have the humility to spend their time wisely because they realize they would be wasting their project stakeholder's resources otherwise. Is this advice contradictory? I don't think so. If something is worth keeping, then it's worth building properly; otherwise, invest minimal effort in creating it.

Rapid Feedback

Feedback is one of the five values of AM, and because the time between an action and the feedback on that action is critical, agile modelers prefer rapid feedback over delayed feedback whenever possible. By working with other people on a model, particularly when you are working with a shared-modeling technology such as a whiteboard, CRC cards, or essential modeling materials such as Post-It notes, you are obtaining near-instant feedback on your ideas. This gives you an indication of whether or not your approaches are likely to solve the situation at hand, as well as provides opportunities to evolve and improve your model(s). Working closely with your customer to understand their requirements, to analyze those requirements, or to develop a user interface that meets their needs provides opportunities for rapid feedback. Writing code based on your models is another opportunity for feedback because it shows whether your model is feasible and very often reveals flaws in your approach because you simply can't think all of the issues through. Obtaining feedback on your work is a humbling but informative experience, one that you want sooner rather than later so you can act on any issues before they become serious problems.

There are two reasons why rapid feedback is important: We make most of our mistakes in the "early" aspects of development and the cost of fixing defects increases exponentially the later they are found. Technical people are very good at technical things such as design and coding—that is why they are technical people. Unfortunately, technical people are often not as good at non-technical tasks such as gathering requirements and performing analysis—probably another reason why they are technical people. The result, as shown in Figure 3.3, is that developers have a tendency to make more errors during requirements definition and analysis than during design and coding. Furthermore, on a non-agile project the cost of fixing these defects rises the later they are found as shown in Figure 3.4. This happens because of the nature of software development—work is performed based on work performed previously. For example: Design modeling is performed based on your requirements. Programming is done based on the design models, and testing is performed on the written source code.

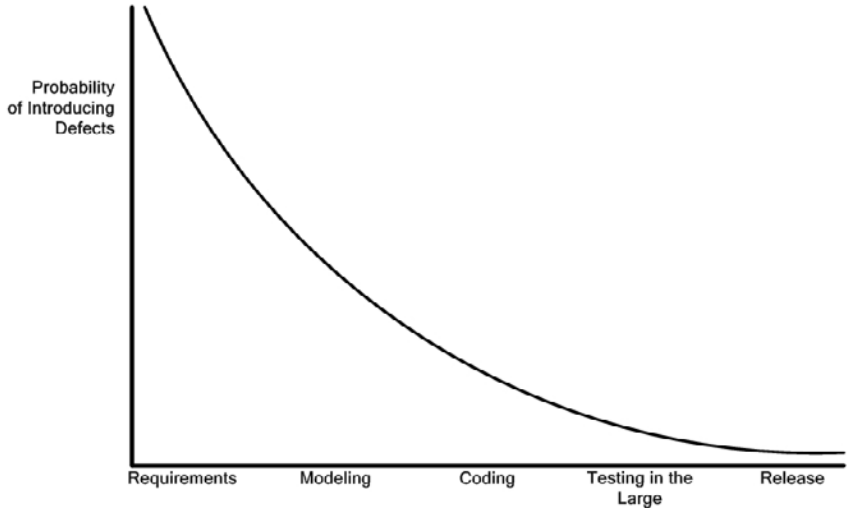


Figure 3.3 The decreasing probability of introducing defects.

If a requirement was misunderstood, all modeling decisions based on that requirement are potentially invalid, all code written based on the models is also in question, and the testing efforts are now verifying the application against the wrong conditions. If the only feedback you receive is the errors detected late in the lifecycle of your project, during testing in the large, or after the application has been released, they are likely to be very expensive to fix. However, if you receive feedback quickly, just after

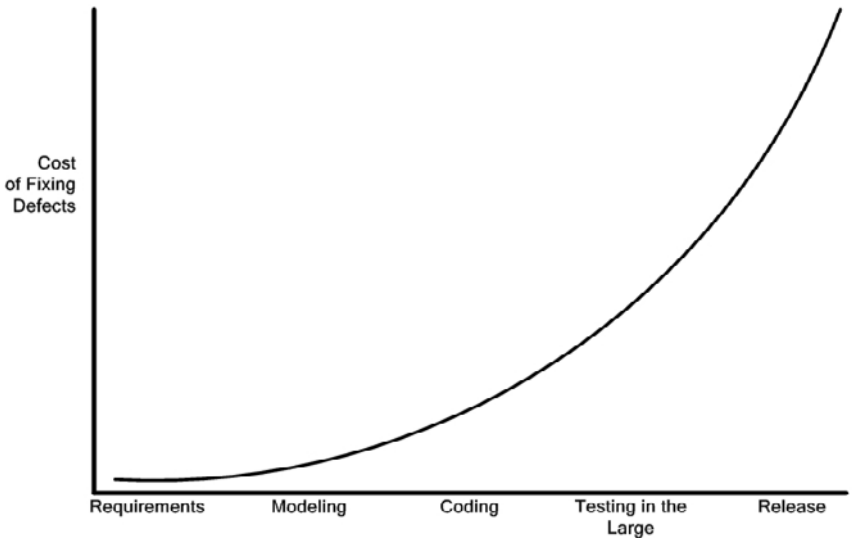


Figure 3.4 The rising costs of finding and fixing defects.

misunderstanding what you were originally told, it will be much less expensive to address the misunderstanding.

Maximize Stakeholder Investment

Your project stakeholders are investing resources—time, money, facilities, and so on—to have software developed that meets their needs. Stakeholders deserve to invest their resources the best way possible and to not have them frittered away by your team. Furthermore, stakeholders deserve to have the final say in how those resources are invested or not invested. If it was your money, would you want it any other way?

TIP System Documentation is a Business Decision, Not a Technical One
It is important to recognize that every time you decide to keep a model or document, you are making a serious trade-off—you are forgoing new functionality in order to write documentation. When you stop and think about it, this is a trade-off that is a business decision, not a technical one. You are trading business functionality for the potential risk-reduction benefits of having permanent artifacts that describe your system. Therefore, you should go to your project stakeholders and ask their permission to invest their resources in this manner, presenting the advantages and disadvantages of doing so. Sometimes they will choose to keep the artifact(s) that you suggest, and other times they will choose to accept the risks of not having them and instead travel light. That's their decision, not yours.

Why Core Principles?

Why do I stress the need to adopt all of AM's core principles to truly claim that you're doing AM? I want to avoid the problem that XP has faced—people who claim to do XP but who really aren't, then blame XP for their failure. Like XP, the principles and practices of AM are synergistic, and if you remove some, the synergy is lost. By failing to adopt one of the core principles or practices of AM, you reduce the method's effectiveness. Yes, you can benefit by only adopting a portion of AM, but you likely won't obtain dramatic improvements in your effectiveness because of the drop in synergy. In short, feel free to adopt whatever aspects of AM that you see fit, just please don't claim that you're doing AM when you've only partially adopted it.